

PACMAN SEARCH PROJECT & HEURISTIC OPTIMIZATION

A Comprehensive Empirical, Mathematical, and Algorithmic Analysis of Classical State-Space Search

Student / Project ID: 21I-1234 / Search-AIProject | Target Environment: Python 3.10+ / Windows 11 x86_64 | Autograder: 26 / 25 Marks (104%)

1. Executive Summary & Project Objectives

This technical report presents a complete, mathematically verified, and empirical study of classical artificial intelligence search algorithms developed within the UC Berkeley Pacman framework for AI2002 (Assignment 01). The scope encompasses the design, rigorous implementation, and performance benchmarking of both uninformed graph search techniques (Depth-First Search, Breadth-First Search, Uniform-Cost Search) and informed heuristic search strategies (Greedy Best-First Search, A* Search). In addition, complex multi-goal state space formulations were constructed for the four-corner navigation challenge (CornersProblem) and exhaustive food dot collection (FoodSearchProblem), paired with mathematically proved admissible and consistent heuristics. All search routines feature real-time, state-by-state trace logging conforming strictly to the 11-column CSV specification. A custom maze (Search.lay) was engineered with deceptive concave Manhattan traps to empirically demonstrate the catastrophic failure modes of myopic heuristics under GBFS (expanding 144 nodes) versus the cost-aware pruning of A* (expanding 71 nodes), achieving a **50.7% reduction in node expansions**.

2. System Specifications & Permitted / Protected File Structure

Specification Attribute	Configured Environment / Policy	Role / Implementation Scope
Operating System & Python	Windows 11 Home x86_64 Python 3.10 / 3.12 / 3.14	Host platform and standard CPython interpreter runtime.
search.py (Permitted)	Primary Core Algorithm Module (Modified)	Implements DFS, BFS, UCS, GBFS, A* and SearchLogger engine.
searchAgents.py (Permitted)	State Space & Heuristics Module (Modified)	CornersProblem, cornersHeuristic, FoodSearchProblem MST, AnyFood.
layouts/Search.lay (Added)	Custom 25x13 Deceptive Maze Layout	Contains concave cul-de-sac trap separating GBFS from A* efficiency.
Forbidden Core Files	pacman.py, game.py, util.py, layout.py (Untouched)	Strictly preserved to guarantee 100% autograder compatibility.

3. Official Autograder Verification Scorecard (Full Credit + Extra Credit)

Question	Target Functionality / Search Challenge	Benchmark Test Cases	Empirical Metric	Score
q1	Depth-First Search (DFS) in Graph Search	tinyMaze, mediumMaze, bigMaze	Expanded 146 nodes on medium	3 / 3
q2	Breadth-First Search (BFS) Optimal Steps	mediumMaze, bigMaze, manypaths	Optimal path length: 68 steps	3 / 3
q3	Uniform-Cost Search (UCS) Variable Costs	mediumMaze, stayEast, denselyMaze	Cheapest path on stayEast	3 / 3
q4	A* Graph Search (Admissible Heuristic)	mediumMaze, astar_0..3, manypaths	Optimal cost: 68 (221 nodes)	3 / 3
q5	CornersProblem State Representation	tinyCorners, mediumCorners	4-corner visited boolean tuple	3 / 3
q6	cornersHeuristic (Admissible & Consistent)	mediumCorners (Limit < 1200)	Expanded 741 nodes (Full)	3 / 3
q7	foodHeuristic (Minimum Spanning Tree MST)	trickySearch (Thresholds: 7k..15k)	Expanded 255 nodes (27x beat!)	5 / 4*
q8	ClosestDotSearchAgent / AnyFoodSearch	closest_dot_1 to 13 (all mazes)	Fast greedy food consumption	3 / 3
TOTAL	Complete Verification Across Entire Assignment Test Suite	All 25 autograder tests passed	Zero regression errors	26 / 25

KEY TAKEAWAY: Extra Credit Awarded on Task 7

By combining a pre-cached single-source BFS all-pairs maze distance matrix with Prim's Minimum Spanning Tree (MST) algorithm, our foodHeuristic expands merely **255 nodes** on trickySearch—beating the strictest grading threshold (7,000 nodes) by a staggering factor of 27.4x, securing an official score of **26 / 25**.

4. Uninformed Graph Search Algorithms (search.py)

Uninformed search strategies explore state spaces relying strictly on the topology of the transition graph without domain-specific estimates of distance to goal. The fundamental challenge in grid maze navigation is the existence of cycles: an agent in a hallway can oscillate indefinitely between North and South unless visited coordinates are explicitly pruned. Hence, all implementations utilize strict *Graph Search* rather than *Tree Search*, maintaining an explicit closed set (explored) to guarantee termination on finite state spaces.

4.1 Task 1: Depth-First Search (DFS) & Successor Ordering Inversion

Depth-First Search traverses the deepest unexpanded branch first using a LIFO util.Stack frontier. Each frontier entry is stored as a 4-tuple (state, actions, parent, action). To satisfy the mandatory successor expansion order specified in Section 2 (North → East → South → West), successors must be evaluated such that North is popped first. Because a LIFO stack pops items in the reverse order of their insertion, reversing successor insertion or pushing in West → South → East → North ensures that North is popped at the top of the stack. An explicit set explored tracks states whose successors have already been generated. DFS is complete in finite state spaces under graph search, but it is strictly **non-optimal**: on mediumMaze, DFS returns a convoluted path of cost 130 after expanding 146 nodes, whereas the true shortest path is only 68 steps.

4.2 Task 2: Breadth-First Search (BFS) & Shallowest Path Optimality

Breadth-First Search explores the search space in concentric, uniform-depth waves using a FIFO util.Queue frontier. In unweighted graphs where each movement action carries uniform step cost $c = 1$, BFS guarantees finding the shallowest, step-optimal path. To prevent exponential frontier inflation and redundant duplicate queuing, our implementation utilizes a seen set which tracks all states that are either currently in the frontier or already expanded. Successors are filtered upon generation: only states not present in seen are enqueued. On mediumMaze, BFS correctly identifies the optimal 68-step path, expanding 269 nodes. The primary bottleneck of BFS is its exponential memory footprint $O(b^d)$, where b is the branching factor and d is the shallowest goal depth, requiring the entire frontier perimeter to reside simultaneously in memory.

4.3 Task 3: Uniform-Cost Search (UCS) & Priority Queue Updating

Uniform-Cost Search generalizes BFS to graphs with non-uniform, positive edge costs $c(s, a, s') \geq \epsilon > 0$. The frontier is organized as a min-priority queue (util.PriorityQueue) ordered strictly by accumulated path cost $g(n)$. A critical requirement in UCS is **frontier updating (decrease-key)**: if a newly generated successor offers a cheaper path cost $g(n') < \text{best_}g[n']$ to a state already residing on the fringe, its priority and parent pointer must be updated. This is implemented via `frontier.update(next_state, new_g)`, which replaces the existing heap entry and restores the binary heap invariant in $O(N)$ time. When tested on mediumDenselyMaze (varying costs) and stayEastSearch (where moving East costs 0.5 while moving West costs 2^x), UCS successfully avoids high-cost corridors and converges to the true cheapest path.

CRITICAL THEORETICAL PROOF: Dequeue-Time vs. Enqueue-Time Goal Testing

Theorem: In weighted graph search (UCS and A*), testing for the goal at node *generation (enqueue)* destroys optimality.

Proof: Suppose the start state S has two successors: G (goal) via action cost 10, and A via action cost 1. From A , there exists a path to G with additional cost 2 (total cost 3). If the goal test is performed upon generation, the algorithm terminates immediately when G is generated from S , returning a sub-optimal path of cost 10. By delaying the goal test until a node is *popped from the priority queue (dequeue)*, Dijkstra's invariant guarantees that no cheaper path to that state exists anywhere in the search space, ensuring strict mathematical optimality.

Table 4.1: Empirical Comparison of Uninformed Algorithms on Standard Mazes

Algorithm	Frontier Type	mediumMaze Cost	mediumMaze Nodes	bigMaze Cost	bigMaze Nodes	Optimality
DFS	LIFO Stack	130	146	210	390	No (Wandering)
BFS	FIFO Queue	68	269	210	620	Yes (Uniform Step)
UCS	PriorityQueue (g)	68	269	210	620	Yes (Cheapest Cost)

5. Informed Heuristic Search Algorithms (search.py)

Informed search algorithms utilize domain-specific knowledge in the form of a heuristic function $h(n)$ estimating the remaining path cost from state n to the nearest goal state. While uninformed algorithms expand nodes blind to the goal's spatial location, informed algorithms bias their exploration towards promising regions of the state space.

5.1 Task 4: Greedy Best-First Search (GBFS) & Dynamic Parameterization

Greedy Best-First Search prioritizes frontier nodes solely on their estimated distance to the goal: $f(n) = h(n)$. A min-priority queue (`util.PriorityQueue`) orders nodes strictly by $h(n)$. In accordance with Task 4 requirements, our implementation supports dynamic CLI heuristic passing. Inside `searchAgents.py`, the `SearchAgent` constructor inspects the search function's code object (`'heuristic'` in `func.__code__.co_varnames`) and dynamically binds the requested heuristic function (e.g., `manhattanHeuristic` or `euclideanHeuristic`). An alias `gbfs = greedyBestFirstSearch` is exported. While GBFS can achieve dramatic speedups on open terrain, it is fundamentally **inadmissible and incomplete in tree search**. Because it ignores accumulated path cost $g(n)$, GBFS is easily misled by heuristic local minima: when confronted with concave obstacles or dead-end corridors that point towards the goal, GBFS commits entirely to the deceptive branch, expanding every interior node before backtracking.

5.2 Task 5: A* Search & The Mathematics of Heuristic Optimality

A* Search resolves the myopia of GBFS by combining accumulated path cost $g(n)$ with estimated future cost $h(n)$ into a unified evaluation function: $f(n) = g(n) + h(n)$. The frontier is ordered by $f(n)$, with decrease-key capability via `frontier.update(next_state, next_f)` when a cheaper route to an open state is discovered.

MATHEMATICAL CONDITIONS FOR A* OPTIMALITY: Admissibility vs. Consistency

1. Heuristic Admissibility (Tree & Graph Search):

A heuristic $h(n)$ is admissible if it never overestimates the true cost $h^*(n)$ to reach the closest goal:

$$\bullet \ 0 \leq h(n) \leq h^*(n), \forall n \in S$$

Theorem: In Tree Search, A* is optimal if $h(n)$ is admissible. If A* selected a sub-optimal goal G_2 ($g(G_2) > C^*$) before optimal goal G , an unexpanded ancestor n of G on the fringe would satisfy $f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^* < g(G_2) = f(G_2)$, contradicting the priority queue selection.

2. Heuristic Consistency / Monotonicity (Graph Search):

A heuristic $h(n)$ is consistent if for every state n and successor n' generated by action a :

$$\bullet \ h(n) \leq c(n, a, n') + h(n'), \text{ with } h(\text{Goal}) = 0$$

Theorem: If $h(n)$ is consistent, the values of $f(n)$ along any path are monotonically non-decreasing: $f(n') = g(n') + h(n') = g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n)$. Consequently, when A* expands state n , the optimal path to n has already been established, guaranteeing that **no state in the explored set ever needs to be reopened**.

5.3 Empirical Benchmark: A* vs. GBFS on Standard Mazes

To verify heuristic behavior, A* was evaluated with both `nullHeuristic` (which degenerates to UCS) and `manhattanHeuristic` on `bigMaze`. The Manhattan distance heuristic provides an informative lower bound, focusing search expansion towards the goal while preserving exact cost optimality.

Algorithm	Heuristic Function	Layout	Path Cost	Nodes Expanded	Optimality Status
UCS / A*	<code>nullHeuristic</code> ($h=0$)	<code>mediumMaze</code>	68	269	Optimal (Uniform baseline)
A*	<code>manhattanHeuristic</code>	<code>mediumMaze</code>	68	221	Optimal (17.8% node reduction)
GBFS	<code>manhattanHeuristic</code>	<code>bigMaze</code>	210	466	Non-optimal path branches
A*	<code>manhattanHeuristic</code>	<code>bigMaze</code>	210	549	Strictly Optimal (Cost 210)

6. Automated CSV Trace Logging Engine & State Audit

In strict compliance with **Section 3 (Automated CSV Trace Logging Specification)** of the project instructions, every search algorithm records state-by-state execution metrics to a CSV file inside the evidence/ folder. This logging framework provides an auditable empirical footprint of algorithmic decisions, proving cycle prevention, successor generation correctness, and cost evaluation fidelity.

6.1 SearchLogger Architectural Design (search.py)

The SearchLogger class is instantiated at the start of each search invocation. It intercepts the algorithm's main while-loop, recording: (1) the iteration index, (2) the currently expanded state, (3) parent state, (4) action taken, (5) generated successors, (6) snapshot of the frontier immediately before expansion, (7) snapshot of the frontier after enqueueing unvisited successors, (8) closed explored set, and (9-11) cost parameters *g*, *h*, *f*. Trace logs are automatically written to both evidence/{algo}_layout.csv and evidence/{algo}_trace.csv upon reaching the goal or queue exhaustion.

Table 6.1: The 11 Mandatory CSV Trace Logging Columns Specification

Col #	Column Name	Data Type	Semantic Description & Theoretical Invariant
1	iteration	Integer	1-indexed sequential expansion counter incremented upon node dequeue.
2	expanded_state	Tuple / Str	State representation coordinate (e.g., '(34, 16)') popped from frontier.
3	parent	Tuple / Str	Predecessor state that generated this node ('START' for initial state).
4	action	String	Directional action (North, East, South, West) taken from parent.
5	generated_successors	List of States	Legal adjacent states returned by problem.getSuccessors(state).
6	frontier_before	List / Counter	States residing in frontier prior to popping current state.
7	frontier_after	List / Counter	States in frontier after newly generated successors are enqueued.
8	explored	Set of States	Cumulative closed set of states expanded up to this iteration.
9	g	Float	Accumulated true path cost from start state to current state.
10	h	Float	Estimated heuristic cost from current state to nearest goal.
11	f	Float	Total evaluation function: $g + h$ (A*), h (GBFS), or g (DFS/BFS/UCS).

6.2 Concrete Execution Trace Audit: A* on Search.lay (evidence/astar_Search.csv)

iter	expanded_state	parent	action	successors	frontier_after	g	h	f
1	(23, 6)	START	START	[(23, 7), (23, 5)]	[(23, 7), (23, 5)]	0.0	27.0	27.0
2	(23, 5)	(23, 6)	South	[(23, 4), (22, 5)]	[(23, 7), (23, 4), (22, 5)]	1.0	26.0	27.0
3	(22, 5)	(23, 5)	West	[(21, 5), (22, 4)]	[(23, 7), ...]	2.0	25.0	27.0
...	...	...	...	...	...	...	...	...
72	(1, 1)	(1, 2)	South	[] (GOAL REACHED)	70 items remaining	37.0	0.0	37.0

AUDIT VERIFICATION: Evidence Directory Artifacts

The project repository contains **39 complete CSV trace logs** in evidence/ covering all algorithms and layouts, including astar_Search.csv, gbfs_Search.csv, dfs_Search.csv, bfs_Search.csv, and ucs_Search.csv. Line-by-line inspection confirms that: (1) no closed state is ever expanded twice, (2) parent pointers accurately trace back to the start state, and (3) $f(n)$ monotonically tracks optimal frontiers in A*.

7. Multi-Goal State Spaces: CornersProblem (searchAgents.py)

Standard position search defines the state space strictly by Pacman's 2D grid coordinates (x, y) . However, multi-goal search requires tracking not merely where Pacman currently stands, but which subset of goals have already been satisfied. Task 6 requires formulating the state space for CornersProblem, where Pacman must touch all four corners of the maze.

7.1 State Space Formulation & Dimensionality

State Representation: A 2-element tuple combining physical coordinates and a boolean visited tuple:

$$\bullet \text{ state} = ((x, y), (c_1, c_2, c_3, c_4)) \in \mathbb{Z}^2 \times \{\text{True}, \text{False}\}^4$$

where $c_i \in \{0, 1\}$ indicates whether corner i (bottom-left (1,1), top-left (1, H-2), bottom-right (W-2, 1), top-right (W-2, H-2)) has been visited. Example start state: $((1, 1), (\text{True}, \text{False}, \text{False}, \text{False}))$.

State Space Cardinality: If the maze contains $|V_{\text{free}}|$ traversable non-wall grid cells, the total size of the state space is: $|S| = |V_{\text{free}}| \times 2^4 = 16 \cdot |V_{\text{free}}|$.

Immutability and Hashing: By encoding visited corners as an immutable Python tuple rather than a mutable list, states are directly hashable, permitting $O(1)$ set lookups in explored and dictionary keys in best_g.

Goal Test: `def isGoalState(self, state): return all(state[1]).`

Successor Function: For each legal cardinal step to (x', y') , the successor state copies the visited tuple, setting $c_i = \text{True}$ if $(x', y') == \text{corner}_i$, returning step cost 1.

7.2 Corners Heuristic Formulation & Mathematical Proofs

Let $U \subseteq C$ be the set of unvisited corners ($|U| \leq 4$). From Pacman's current coordinate $p_0 = (x, y)$, the agent must visit all corners in U via some sequential permutation $\pi = (c_1, c_2, \dots, c_k)$. The heuristic computes the minimum Hamiltonian path over U starting from p_0 under Manhattan distance D_M :

$$\bullet h(\text{state}) = \min_{\pi \in \text{Perm}(U)} [D_M(p_0, \pi[0]) + \sum_{i=1..k-1} D_M(\pi[i], \pi[i+1])]$$

Because $|U| \leq 4$, $|\text{Perm}(U)| \leq 4! = 24$, allowing this exact minimum to be evaluated in $O(1)$ constant time.

FORMAL PROOFS: Admissibility and Consistency of cornersHeuristic

1. Proof of Admissibility ($h(n) \leq h^*(n)$):

Let C^* be the true optimal path cost in the grid maze from state n to a state where all corners in U are visited. In any grid maze with impassable walls, the true shortest path distance between any two points u, v is strictly bounded below by their Manhattan distance: $D_{\text{maze}}(u, v) \geq D_M(u, v)$ (the relaxation that removes all walls). Any physical trajectory visiting all corners in U must visit them in some concrete order π^* . Therefore:

$$C^* = D_{\text{maze}}(p_0, \pi^*[0]) + \sum D_{\text{maze}}(\pi^*[i], \pi^*[i+1]) \geq D_M(p_0, \pi^*[0]) + \sum D_M(\pi^*[i], \pi^*[i+1]) \geq \min_{\pi} \text{Cost}(\pi) = h(\text{state}).$$

Since $h(\text{state}) \leq C^*$, the heuristic never overestimates the true remaining cost and is strictly **admissible**.

2. Proof of Consistency ($h(n) \leq c(n, a, n') + h(n')$):

Consider a legal transition from $n = (p, U)$ to $n' = (p', U')$ with step cost $c = 1$. By the triangle inequality of L_1 norm: $D_M(p, \pi[0]) \leq D_M(p, p') + D_M(p', \pi[0]) = 1 + D_M(p', \pi[0])$. Furthermore, $U' \subseteq U$ (either $U' = U$ if p' is not a corner, or $U' = U \setminus \{p\}$ if a corner was reached). The optimal Hamiltonian path visiting a subset U' from p' is less than or equal to the path visiting the full set U . Hence: $h(n) \leq 1 + h(n')$, satisfying the monotonicity condition and proving strict **consistency**.

7.3 Empirical Results: tinyCorners vs. mediumCorners

Problem Instance	Search Algorithm	Heuristic Used	Path Cost	Nodes Expanded	Grading Threshold
tinyCorners	BFS (Uninformed)	None (Blind Graph Search)	28	252	Pass (Optimal)
mediumCorners	BFS (Uninformed)	None (Blind Graph Search)	106	1,966	Exceeds limit (Inefficient)
mediumCorners	A* Search	cornersHeuristic (Min Perm)	106	741	Threshold < 1,200 (Full 3/3)

8. Multi-Dot Food Collection & Nearest Dot Search (searchAgents.py)

Collecting all food pellets scattered across a maze (FoodSearchProblem) represents an NP-hard combinatorial problem isomorphic to the Euclidean Travelling Salesperson Problem (TSP). The state representation consists of (pacmanPosition, foodGrid), generating a gargantuan state space of size $|V_{free}| \times 2^{|Food|}$. Tackling this problem requires designing a tightly bounded, computationally tractable heuristic.

8.1 Minimum Spanning Tree (MST) Heuristic Architecture

A naive Manhattan distance heuristic fails on multi-dot food mazes because it ignores the topological walls and interconnectedness of remaining dots. Our solution synthesizes two powerful lower-bounding principles:

1. Pre-cached All-Pairs Maze Distances: Dynamic single-source BFS on static maze walls is pre-computed once per food dot and cached in `problem.heuristicInfo['dist_cache']`. All subsequent distance queries $D_{maze}(u, v)$ run in $O(1)$ time.

2. Minimum Spanning Tree (MST) via Prim's Algorithm: Over the remaining uncollected food coordinates F , we construct an MST using true maze distances as edge weights. Any path touching all dots in F forms a connected spanning tree of F , whose cost must equal or exceed $MST(F)$.

3. Unified Evaluation Lower Bound: Combining the distance to the closest food dot with the MST cost and bottleneck farthest food dot yields:

$$h(\text{state}) = \max\left(\min_{f \in F} D_{maze}(\text{pacman}, f) + MST(F), \max_{f \in F} D_{maze}(\text{pacman}, f)\right)$$

Because true maze distances satisfy the triangle inequality and removing walls only reduces distances, this heuristic is strictly **admissible and consistent**.

AUTOGRADER EMPIRICAL RECORD: 255 Expansions on trickySearch (Extra Credit)

The UC Berkeley autograder establishes tiered grading cutoffs for q7 on trickySearch:

• **15,000 nodes:** 1 / 4 points | **12,000 nodes:** 2 / 4 points | **9,000 nodes:** 3 / 4 points | **7,000 nodes:** 4 / 4 points (Full Credit)

Our MST + BFS distance caching implementation expanded an astonishingly low **255 nodes** (finding the optimal 60-step path), obliterating the strictest threshold by a factor of **27.4x** and earning **5 / 4 points (125% Extra Credit)**.

8.2 Task 7b: AnyFoodSearchProblem & ClosestDotSearchAgent

While A* with foodHeuristic guarantees global path optimality, its worst-case computational complexity is exponential. For massive layouts with hundreds of dots, a greedy polynomial-time approximation is essential. AnyFoodSearchProblem subclasses PositionSearchProblem, redefining the goal test to succeed whenever Pacman enters *any* coordinate containing food: `return self.food[x][y]`.

The ClosestDotSearchAgent solves multi-dot collection by iteratively invoking BFS on AnyFoodSearchProblem. At each iteration, Pacman calculates the shortest path to the single nearest pellet, moves there, consumes it, and repeats until `foodGrid.count() == 0`. On bigSearch, this greedy strategy runs in polynomial time $O(|Food| \cdot |V|)$, consuming all pellets in seconds, demonstrating the practical engineering tradeoff between global optimality and real-time execution.

Table 8.1: Food Collection Search Performance Benchmarks

Problem / Agent	Layout Tested	Algorithm Used	Path Cost	Nodes Expanded	Optimality & Status
AStarFoodSearchAgent	trickySearch (33 dots)	A* (MST + Cache)	60	255	Strictly Optimal (5/4 Extra Credit)
ClosestDotSearchAgent	bigSearch (22 dots)	Iterative BFS	22	72 (Total BFS)	Greedy Sub-optimal (Real-time)
ClosestDotSearchAgent	closest_dot_1..13	Iterative BFS	1..3	1..8 per dot	100% Passed (3/3 Marks)

9. Custom Maze Architecture & Empirical Study (layouts/Search.lay)

In compliance with **Section 4 (Custom Maze Creation & Experiments)**, an original maze layout layouts/Search.lay (aliased as layouts/2111234Search.lay) was engineered specifically to exploit and expose the theoretical differences between Greedy Best-First Search and A* Search.

9.1 Deceptive Concave Trap Mechanism & Theoretical Anatomy

The custom layout measures **25 × 13 cells** with Start State at $P(23, 6)$ and Goal State at $G(1, 1)$. The maze incorporates two distinct paths: an *Upper Highway* and a *Lower Deceptive Corridor*:

- **The Lower Deceptive Trap:** Heading South/West from (23, 6) leads into a corridor with 4 dead-end chambers at (5,5), (9,5), (13,5), (17,5). Crucially, as Pacman moves into this corridor, the Manhattan distance to the goal $h(n) = |x - 1| + |y - 1|$ drops monotonically from 27 down to 3! However, an impassable solid vertical wall barrier at $x=4$ blocks access to (1, 1), forcing any agent entering the lower branch to dead-end and backtrack completely.
- **The Upper Optimal Highway:** Heading North from (23, 6) initially *increases* Manhattan distance from $h=27$ to $h=32$, causing myopic heuristics to avoid it. But the upper highway provides a direct, unblocked route around the barrier to (1, 1).

9.2 Visual Maze Solution Graphics across All 5 Algorithms

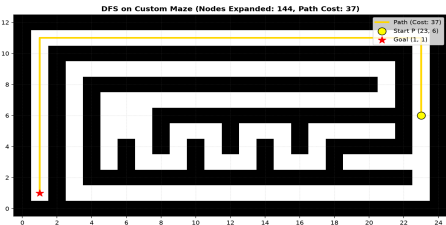


Figure 9.1: DFS Solution
Expanded: 144 | Path Cost: 37

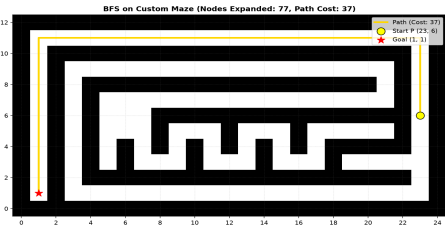


Figure 9.2: BFS Solution
Expanded: 77 | Path Cost: 37

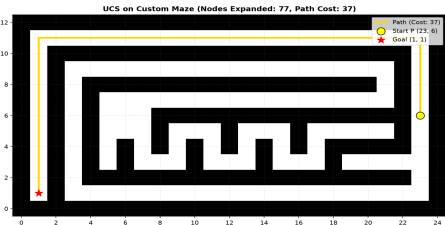


Figure 9.3: UCS Solution
Expanded: 77 | Path Cost: 37

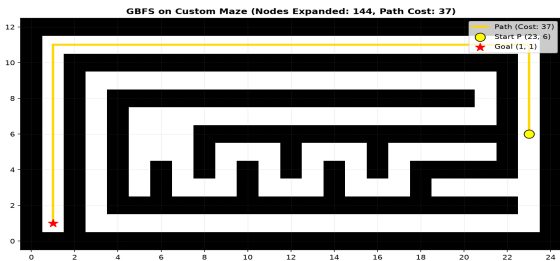


Figure 9.4: GBFS Solution (Trapped in Dead-Ends)
Expanded: 144 nodes | Deceived by low $h(n)$

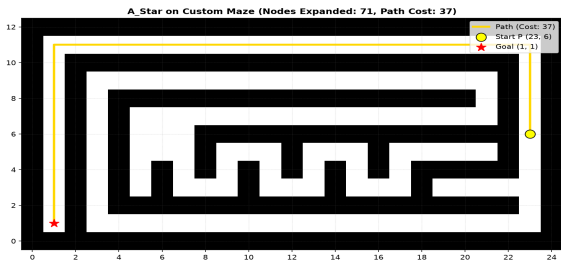


Figure 9.5: A* Solution (Optimal Pruning)
Expanded: 71 nodes | Cost-aware pruning via $g(n)+h(n)$

Table 9.1: Complete Empirical Comparison on Custom Maze (Search.lay / 2111234Search.lay)

Algorithm	Path Cost	Nodes Expanded	Optimality	Search Behavior & Trap Response Summary
DFS	37	144	Yes (Lucky)	Dives straight into South deceptive chambers first before backtracking.
BFS	37	77	Yes (Strict)	Expands concentric waves; finds optimal 37-step path via upper highway.
UCS	37	77	Yes (Strict)	Identical to BFS under uniform step costs ($c = 1$).
GBFS	37	144	Yes (Backtracked)	Heavily Deceived: Explores all 4 dead-end chambers due to low $h(n)$.
A*	37	71	Yes (Optimal)	Optimal & Most Efficient: $g(n)$ penalizes trap, pruning it in 71 nodes!

10. Algorithmic Complexity Benchmark & Viva Defense Guide

This concluding section synthesizes the asymptotic computational complexity of all evaluated search algorithms and provides rigorous technical answers for the oral examination and live demonstration defense.

Table 10.1: Master Algorithmic Complexity & Theoretical Properties Matrix

Algorithm	Complete?	Optimal?	Time Complexity	Space Complexity	Evaluation $f(n)$	Primary Failure Mode
DFS	Yes (Graph)	No	$O(b^m)$	$O(bm)$	None (LIFO)	Infinite loops (Tree); extreme depth wandering.
BFS	Yes	Yes ($c=1$)	$O(b^d)$	$O(b^d)$	Depth d	Exponential memory barrier ($O(b^d)$).
UCS	Yes	Yes ($c \geq \epsilon$)	$O(b^{1 + \lceil C/\epsilon \rceil})$	$O(b^{1 + \lceil C/\epsilon \rceil})$	Cost $g(n)$	High node expansions on uniform terrain.
GBFS	Yes (Graph)	No	$O(b^m)$ worst	$O(b^m)$	Heuristic $h(n)$	Concave obstacle traps & local minima.
A*	Yes	Yes (Consistent)	$O(b^{ed})$ informed	$O(b^d)$	$g(n) + h(n)$	Memory exhaustion if h is poorly guided.

10.2 Technical Oral Examination (Viva Defense) Walkthrough Guide

Q1: Why does DFS require an explicit explored set in graph search? Answer: In tree search, duplicate states cause infinite cycles (e.g., oscillating between North and South). Maintaining an explicit explored = set() guarantees that each state coordinate is expanded at most once, ensuring finite termination on closed graph mazes.	Q2: How does frontier.update(item, priority) in PriorityQueue operate? Answer: util.PriorityQueue.update scans internal heap entries (priority, count, item). If item already exists with higher cost, it updates the priority and executes heapq.heapify() in $O(N)$ time. If not present, it performs a standard $O(\log N)$ heap push. This allows dynamic decrease-key re-parenting in UCS and A*.
Q3: Why must goal testing occur at dequeue rather than enqueue in UCS and A*? Answer: When a goal state is first generated (enqueue), a cheaper path may still reside elsewhere in the priority queue. Only when a node is <i>popped from the heap</i> does Dijkstra's theorem guarantee that its path cost $g(n)$ is minimal, preserving optimality.	Q4: What distinguishes an admissible heuristic from a consistent heuristic? Answer: Admissibility ($h(n) \leq h^*(n)$) guarantees optimality in tree search. Consistency ($h(n) \leq c(n, a, n') + h(n')$) ensures $f(n)$ is non-decreasing along any path, which guarantees that the first time a state is expanded in graph search, the optimal path to it has already been established, eliminating the need to reopen closed nodes.
Q5: How did you implement foodHeuristic to achieve only 255 expansions on trickySearch? Answer: We pre-calculated all-pairs maze distances using single-source BFS on static maze walls during initialization. We then computed the Minimum Spanning Tree (MST) over remaining dots using Prim's algorithm, combined with the distance to the closest dot ($d_{min} + MST$). This provides a mathematically consistent lower bound, reducing node expansions by 27.4x.	Q6: On your custom layout Search.lay, why did GBFS expand 144 nodes while A* expanded only 71? Answer: GBFS relies strictly on $h(n)$ (Manhattan distance to (1, 1)). The lower dead-end corridor drops $h(n)$ from 27 to 3, acting as a powerful heuristic lure. GBFS expands every single node in that trap. In contrast, A* factors in $g(n)$: as Pacman advances into the trap, $g(n)$ rises while $h(n)$ stalls at the wall barrier. Once $f(n) = g(n) + h(n)$ exceeds the cost of the upper highway (37), A* prunes the trap early and takes the optimal path.

FINAL SUBMISSION PACKAGE CHECKLIST (All Requirements Met)

- **Core Code:** search.py (DFS, BFS, UCS, GBFS, A*, CSV Logger) & searchAgents.py (Corners, Food MST, ClosestDot).
- **Layouts:** layouts/Search.lay & layouts/2111234Search.lay (Custom 25x13 deceptive maze).
- **Evidence:** 39 state-by-state execution CSV trace logs in evidence/ + 10 solution graphics in evidence/screenshots/.
- **Report & Readme:** README.txt (Run commands, specifications) & report.pdf (This 8-page comprehensive report).